2025 IEEE 10th International Conference on Smart Cloud (SmartCloud) | 979-8-3315-9665-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/SmartCloud66068.2025.00009

Privacy-Preserving Prompt Injection Detection for Smart Cloud-Deployed Large Language Models

Guozhi Hao[1], Jun Wu[1*]
[1]Graduate School of Information, Production and Systems, Waseda University, Japan
[*]Corresponding Author (Email: junwu@aoni.waseda.jp)

***Abstract*—Large Language Models (LLMs) have recently gained popularity for tasks like text generation and automated reasoning, often hosted on cloud servers to handle large-scale user interactions. Yet this setup faces a critical challenge, prompt injection attacks, where adversaries craft malicious text to bypass an LLM's safety constraints and policy filters. Attackers constantly evolve new injection tactics, rendering static defenses ineffective. While service providers may wish to store user inputs for security audits, this practice poses serious privacy and data-protection risks. User prompts can contain personal or confidential information subject to strict regulatory requirements. To address these conflicting demands of security monitoring and user privacy, we propose a privacy-preserving prompt injection detection scheme via hash and embedding-based auditing for smart cloud-deployed LLMs. Our approach segments each user prompt into smaller chunks and stores only non-invertible representations of them—cryptographic hashes for exact matches and vector embeddings for near-match detection. This enables robust identification of both verbatim and paraphrased threats without retaining any plaintext data. Furthermore, newly discovered injection patterns can trigger retrospective audits on previously stored chunks, greatly enhancing attack traceability. We also introduce an optional blockchain-based logging mechanism for tamper-proof event records. Empirical results show that our method achieves high detection accuracy against diverse injection techniques while safeguarding sensitive user information.**

***Index Terms*—large language models (LLMs), prompt injection, privacy-preserving, blockchain, smart cloud.**

I. Introduction

Large Language Models (LLMs) have recently demonstrated remarkable capabilities in tasks such as creative writing, code generation, and sophisticated reasoning [1], [2]. Increasingly, these models are being deployed on smart cloud servers, which offer elasticity in resource allocation and streamlined global availability. Such a cloud-based setup enables massive throughput and real-time interactions, attracting a growing user base for diverse applications ranging from customer service chatbots to automated tutoring platforms. However, this rapid adoption of cloud-hosted LLMs also exposes critical vulnerabilities. Chief among them is the phenomenon of prompt injection attacks [3]–[5], in which adversaries craft specialized text inputs (often called "injection templates") to circumvent the model's safety alignment and policy constraints. These malicious prompts can coerce an LLM into generating unauthorized or even explicitly illegal content. More concerning, the speed at which new injection methods emerge far outstrips static defenses: once an attacker unveils a novel strategy, existing alignment filters may be easily bypassed until security teams devise countermeasures. During this gap, large-scale LLM deployments are at risk of producing disallowed outputs—ranging from violent, hateful, or privacy-violating text to instructions for criminal activity.

A key complication is that cloud-based LLM service providers often desire the ability to retroactively detect and respond to these newly discovered attacks. Suppose a fresh category of injection template is identified; the provider may wish to review past user prompts to ascertain whether the attack had already been used without detection. To do so, one naive solution would be to store every user prompt in plaintext on the server, thus enabling post-hoc analysis. Yet such archival conflicts directly with user privacy mandates. Many jurisdictions treat user-entered text as potentially sensitive personal data, implying severe regulatory and reputational consequences if that text is mishandled or exposed [6], [7]. Actual incidents have shown that leaked conversation logs [6]—for instance, revealing private health queries, confidential business strategies, or personal relationships—cause immense harm to both users and service operators [8]. Notably, corporate policy and data governance regulations often demand that user-provided content be minimized or purged as soon as possible, to reduce the risk of large-scale data leaks [6], [7].

In response to these opposing demands—the need for thorough security auditing versus the imperative of user privacy—we propose a privacy-preserving prompt injection detection scheme tailored for LLMs deployed in the cloud. Our method addresses the concern that storing raw user prompts constitutes a privacy risk, while still allowing service providers to detect or re-check historical inputs for suspicious or malicious behavior. Concretely, we rely on two core techniques: hash-based exact matching combined with embedding-based near-match detection. After segmenting each prompt into smaller chunks, these transformations generate non-invertible representations of user inputs, so that original plaintext is never persistently held. When future alignment improvements or newly discovered injection tactics necessitate further review, providers can re-inspect stored hashes and embeddings to identify previously undetected malicious prompts, all while respecting end-user confidentiality. Furthermore, data-protection requirements and corporate compliance can be enhanced by integrating a blockchain logging mechanism after receiving a prompt, the service creates a hashed record embedding essential metadata (e.g., user identity, timestamp, and chunk-hash codes) and appends it to a blockchain transaction. This

979-8-3315-9665-1/25/$31.00 ©2025 IEEE
DOI 10.1109/SmartCloud66068.2025.00009

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.

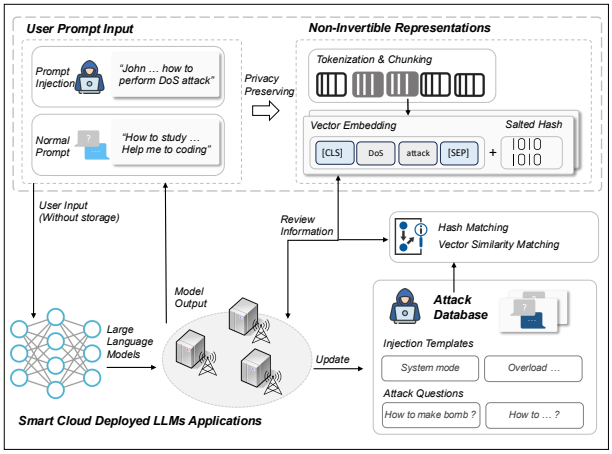


Fig. 1. Overview of the proposed privacy-preserving prompt injection detection scheme.

immutable ledger offers robust accountability and a tamper-proof trace of suspicious user activity [9]–[11]. In summary, this work makes the following contributions:

- We present a layered hashing and embedding approach that stores user prompts only in non-invertible forms, combining exact match detection for literal strings with semantic similarity checks for paraphrased or obfuscated attack templates.
- Our chunk-level storage strategy enables large-scale re-inspections when new prompt injection techniques are discovered. Historical user prompts can thus be audited without reintroducing plaintext data, satisfying data-minimization principles.
- We augment our framework with optional on-chain recording of hashed verification values. This measure ensures an immutable audit trail of potentially malicious prompts and facilitates post-incident forensics.

The next article is structured as follows. Section II presents the scheme overview and the detailed contents of proposed scheme. Implementation and performance evaluation are performed in Section III. Section IV concludes the paper.

II. Proposed Privacy-Preserving Prompt Injection Detection Scheme

In this section, we present a detailed and rigorous approach for detecting prompt injection attacks in a privacy-preserving manner. We first introduce the scheme overview, and then we begin by formalizing the notion of injection templates and attack questions. We provide a series of definitions and equations to illustrate our methodology for hashing, embedding, chunk-based matching, and final classification.

A. Scheme Overview

As shown in Fig. 1, the proposed scheme ensures privacy-preserving prompt injection detection by combining non-invertible data transformations (hashing and vector embeddings), chunk-based analysis, and an optional blockchain-based logging mechanism for tamper-proof auditing. First, the user composes a raw prompt and transmits it to the LLM service over a secure channel (*e.g.*, TLS). The service then partitions the prompt into overlapping chunks of fixed window size W and stride S, limiting any plaintext storage to transient memory usage only. Each chunk x_i is mapped to $(H(x_i), \mathbf{v}_{x_i})$, where $H(\cdot)$ is a salted cryptographic hash function (e.g., SHA256 with a secret salt), preventing recovery of x_i, $\mathbf{E}(\cdot)$ is a pre-trained transformer-based embedding that captures semantic information while remaining largely non-invertible. Afterward, the system detects potential injection templates or malicious questions by:

1) Exact-Match Check (via Hash): comparing $H(x_i)$ against a repository of known injection or attack hashes, identifying verbatim occurrences.
2) Near-Match Check (via Embeddings): comparing $\mathbf{v}_{x_i}$ to stored template/question embeddings via cosine similarity, covering paraphrased or partially modified threats.

If both injection templates and attack questions are detected among the chunks ("*BOTH*"), the prompt is flagged for possible prompt injection. Otherwise, the prompt is labeled as "*NONE*," "*TEMPLATE*," or "*QUESTION*" based on the aggregated chunk-level indicators. Simultaneously, an optional blockchain logging step appends a tamper-proof record. The system composes a log entry that includes user identity (or pseudonym) and timestamp, then computes a log hash $\text{LogHash} = H(\text{ID}_U \| \tau \| \ldots)$, and submits it to the blockchain as a simple transaction. This ensures an immutable audit trail linking user prompts to specific timestamps, without disclosing any plaintext. Hence, historical user actions remain verifiable and traceable, while the stored data itself (hashes and embeddings) protects user privacy in compliance with regulations and internal policy.

B. Preliminaries and Definitions

1) Definition 1 (Prompt Injection Attack): Let $\mathcal{T} = \{T_1, T_2, \ldots, T_m\}$ be a set of known injection templates. Each T_i is a text segment carefully designed to circumvent a LLM safety alignment or policy constraints (e.g., "*Ignore all previous instructions...*"). Similarly, let $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ be a set of known attack questions, each representing a request with clearly malicious or illegal content (e.g., instructions on hacking techniques or how to produce banned substances). We say a user input $\mathbf{x}$ constitutes a prompt injection attack if $\mathbf{x}$ contains—either in sequence or partially interleaved—both at least one injection template from $\mathcal{T}$ and at least one attack question from $\mathcal{Q}$, or semantically close variants thereof. Formally,

$$\exists\, T_i \in \mathcal{T},\, Q_j \in \mathcal{Q} \quad \text{such that} \quad \mathbf{x} \supset (T_i \oplus Q_j), \qquad (1)$$

where $\oplus$ denotes either direct concatenation or textual embedding of the malicious request into the template in a manipulative manner.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.

2) Privacy Requirement: The fundamental requirement in our design is to ensure the system does not store or transmit the raw text $\mathbf{x}$, nor any large portion of it, in a recoverable form. Instead, we keep only non-invertible transformations of $\mathbf{x}$. Concretely, we adopt a salted hash function: $H(\mathbf{x}) = \mathrm{SHA256}\big(\mathrm{salt} \,\|\, \mathbf{x}\big)$, where salt is a secret random string updated periodically to mitigate offline rainbow-table or brute-force attacks. We further define a text embedding function, $\mathbf{E}(\mathbf{x}) \in \mathbb{R}^d$, mapping the user prompt into a d-dimensional vector space via a pre-trained transformer model (e.g., Sentence-BERT). These embeddings are not designed to be inverted back to plaintext, especially if we apply additional noise or quantization steps. In this paper, we primarily present the framework and concepts for detection, without discussing detailed aspects of privacy scheme extensively.

3) Similarity Computation: As malicious users often rephrase templates or questions, we rely on near-match detection in the embedding space. For two embeddings $\mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^d$, we use the cosine similarity: $\mathrm{Sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|}$, and define:

$$\mathrm{isMatch}\big(\mathbf{v}_1, \mathcal{V}, \theta\big) = \begin{cases} 1, & \text{if } \max_{\mathbf{u} \in \mathcal{V}} \mathrm{Sim}(\mathbf{v}_1, \mathbf{u}) \geq \theta, \\ 0, & \text{otherwise,} \end{cases} \tag{2}$$

where $\mathcal{V}$ is a set of stored embeddings (for example, all templates or all attack questions), and θ is a chosen threshold. Lowering θ yields fewer false negatives at the cost of more false positives, while raising θ does the opposite. In practice, θ can be tuned based on validation data.

4) Threat Model: We assume the system is protected so that malicious adversaries cannot directly read the stored embeddings or hashes. Even if partial embeddings or hashes are leaked, the salted hashing reduces feasibility of direct reconstruction of $\mathbf{x}$. The system can be extended to further enhance security (e.g., with secure enclaves or homomorphic encryption), but in this work we focus on the fundamental design for large-scale LLM usage logs.

C. Chunk-Based Matching and Classification

Although some malicious prompts directly concatenate a template T_i with an attack request Q_j, others may embed them deeply within a larger textual input. Hence, scanning the user prompt $\mathbf{x}$ in smaller segments (chunks) reduces the chance of missing disguised or partially inserted fragments. Let $\mathbf{x}$ be a user prompt of length L tokens. We define:

$$\mathrm{Chunk}(\mathbf{x}) = \{x_1, x_2, \ldots, x_K\}, \tag{3}$$

where each x_i is a contiguous sub-prompt of fixed size W tokens. We slide a window of width W across the tokenized prompt with step size S. Hence,

$$K = \left\lceil \frac{L-W}{S} \right\rceil + 1, \tag{4}$$

assuming $L \geq W$. Each x_i is then hashed $H(x_i)$ and embedded $\mathbf{v}_{x_i} = \mathbf{E}(x_i)$. We store two sets of embeddings:

$$\mathcal{V}_T = \{\mathbf{E}(T_1), \ldots, \mathbf{E}(T_m)\}, \mathcal{V}_Q = \{\mathbf{E}(Q_1), \ldots, \mathbf{E}(Q_n)\}.$$

Additionally, we keep the salted hash values $\{H(T_j)\}$ and $\{H(Q_j)\}$. For a sub-prompt x_i, define the binary indicator of template match:

$$\mathrm{isTemplate}(x_i) = \begin{cases} 1, & \text{if } H(x_i) \in \{H(T_j)\}, \\ 1, & \text{if } \mathrm{isMatch}\big(\mathbf{v}_{x_i}, \mathcal{V}_T, \theta_T\big) = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Similarly, the binary question match indicator is:

$$\mathrm{isQuestion}(x_i) = \begin{cases} 1, & \text{if } H(x_i) \in \{H(Q_k)\}, \\ 1, & \text{if } \mathrm{isMatch}\big(\mathbf{v}_{x_i}, \mathcal{V}_Q, \theta_Q\big) = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

Exact hashing detects verbatim or lightly mutated copies of known prompts, whereas embedding-based similarity captures paraphrased or partially obfuscated forms.

Once we process all sub-prompts $x_1, \ldots, x_K$, we sum the matches:

$$\begin{aligned} \mathrm{temp_score}(\mathbf{x}) &= \sum_{i=1}^{K} \mathrm{isTemplate}(x_i), \\ \mathrm{q_score}(\mathbf{x}) &= \sum_{i=1}^{K} \mathrm{isQuestion}(x_i). \end{aligned} \tag{7}$$

We then assign $\mathbf{x}$ to one of four categories based on $(\mathrm{temp_score}, \mathrm{q_score})$:

- NONE if $\mathrm{temp_score}(\mathbf{x}) = 0$ and $\mathrm{q_score}(\mathbf{x}) = 0$,
- TEMPLATE if $\mathrm{temp_score}(\mathbf{x}) > 0$ and $\mathrm{q_score}(\mathbf{x}) = 0$,
- QUESTION if $\mathrm{temp_score}(\mathbf{x}) = 0$ and $\mathrm{q_score}(\mathbf{x}) > 0$,
- BOTH if $\mathrm{temp_score}(\mathbf{x}) > 0$ and $\mathrm{q_score}(\mathbf{x}) > 0$.

Hence, we can detect whether the user input $\mathbf{x}$ is free of suspicious content, contains only an injection attempt, only an attack request, or is a full-fledged prompt injection attack that integrates both.

D. Privacy-Preserving Storage and Auditing

Instead of storing raw prompts, we record only $(H(x_i), \mathbf{v}_{x_i})$ for each chunk x_i. Since both cryptographic hashes and vector embeddings with noise are intentionally non-invertible, the system satisfies strong privacy guarantees [12], [13]. For instance, storing $H(x_i)$ identifies exact textual duplicates to known malicious strings without requiring $\mathbf{x}$ itself. The embedding $\mathbf{v}_{x_i}$ with differential privacy noise permits near-match detection even if the attacker paraphrases or partially hides the template/attack question. At no point do we store $\mathbf{x}$ in a recoverable plaintext form, thus preventing unauthorized exposure of user data.

Furthermore, if the sets $\mathcal{T}$ or $\mathcal{Q}$ are updated (e.g., newly discovered injection templates), the system can re-run detection on historical $(H(x_i), \mathbf{v}_{x_i})$ pairs to identify previously unknown threats. This means we can retroactively label older user inputs without needing the original plaintext $\mathbf{x}$. As soon as a new user prompt arrives, we tokenize and chunk the prompt into sub-prompts of size W with step S. Compute $H(x_i)$ and $\mathbf{v}_{x_i}$ for each chunk, storing them in a protected repository (no

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Privacy-Preserving Prompt Injection Detection Algorithm

ProposedDetection

Data: $\mathcal{T} = \{T_1, \ldots, T_m\}$: Injection templates; $\mathcal{Q} = \{Q_1, \ldots, Q_n\}$: Known attack questions; $\mathbf{x}$: New user prompt to be classified

Result: Label of $\mathbf{x}$ in {NONE, TEMPLATE_ONLY, QUESTION_ONLY, BOTH}

foreach $T_i \in \mathcal{T}$ **do**
  $h_{T_i} \leftarrow H(T_i)$; $\mathbf{v}_{T_i} \leftarrow \mathbf{E}(T_i)$; *Insert into TemplateIndex*;
end
foreach $Q_j \in \mathcal{Q}$ **do**
  $h_{Q_j} \leftarrow H(Q_j)$; $\mathbf{v}_{Q_j} \leftarrow \mathbf{E}(Q_j)$; *Insert into QuestionIndex*;
end
Store $\{(h_{T_i}, \mathbf{v}_{T_i})\}$ and $\{(h_{Q_j}, \mathbf{v}_{Q_j})\}$ only, no plaintext retained; $x_1, \ldots, x_K \leftarrow \mathrm{Chunk}(\mathbf{x}, W, S)$ % window size W, stride S **foreach** x_i **do**
  $h_{x_i} \leftarrow H(x_i)$; $\mathbf{v}_{x_i} \leftarrow \mathbf{E}(x_i)$;
  if $h_{x_i} \in \{h_T\}$ or $\mathrm{isMatch}(\mathbf{v}_{x_i}, \mathcal{V}_T, \theta_T)$ **then** *Flag TemplateMatch* ;
  if $h_{x_i} \in \{h_Q\}$ or $\mathrm{isMatch}(\mathbf{v}_{x_i}, \mathcal{V}_Q, \theta_Q)$ **then** *Flag QuestionMatch* ;
end
Compute $\mathrm{temp_score}(\mathbf{x})$ and $\mathrm{q_score}(\mathbf{x})$ as per (7);
if $\mathrm{temp_score} = 0 \wedge \mathrm{q_score} = 0$ **then** *return NONE*; // NONE
else if $\mathrm{temp_score} > 0 \wedge \mathrm{q_score} = 0$ **then** *return TEMPLATE*; // TEMPLATE_ONLY
else if $\mathrm{temp_score} = 0 \wedge \mathrm{q_score} > 0$ **then** *return QUESTION*; // QUESTION_ONLY
else *return BOTH*; // BOTH

plaintext retained). Perform near-match detection by searching $\mathbf{v}_{x_i}$ against the template set $\mathcal{V}_T$ and the question set $\mathcal{V}_Q$, checking also for exact hash matches. Aggregate the indicators across all chunks as per (7) and classify the prompt into {NONE, TEMPLATE, QUESTION, BOTH}.

At any later time, if new malicious patterns become known, steps 3–4 can be re-invoked on stored $(H(x_i), \mathbf{v}_{x_i})$ pairs to flag previously unidentified suspicious content. Although embeddings are difficult to invert for arbitrary text, advanced attacks (e.g., embedding inversion or membership inference) are possible in principle. To further reduce risk, the system can apply random noise or dimensionality reduction to $\mathbf{v}_{x_i}$, making reconstruction attempts harder. Rotate salts for hashing periodically and discard older salted hashes to invalidate old collisions. Restrict indexing and retrieval so that only authorized modules can query vector similarities, mitigating the risk of embedding-based enumeration. In practice, these measures can be tailored to the operational environment and threat model. The core algorithmic steps described above are captured succinctly in Algorithm 1, which details the phases of (i) *Preprocessing* for storing known template/question embeddings, (ii) *Secure Storage* of hashed data, (iii) *Chunking* of user prompts, (iv) *Sub-prompt Detection*, and (v) *Aggregation & Classification*. This end-to-end process ensures that all detection logic occurs within a privacy-preserving framework, without ever exposing the raw user input outside the local memory scope used for tokenization and embedding.

TABLE I
REAL-WORLD ADDRESS OF BLOCKCHAIN COMPONENTS IN SIMULATION

Address in Ethereum Sepolia Network			
Account of LLM smart cloud 0x46211ab05A523d936A92D54294ff72067f035BD0			
Account of log receiver 0xB56Eb78809F7ae847D6C817Edc20f90D7f9414de			
Transaction address for prompt recording 0x34ee4e95658f9f2be0fd9eb99959e9ba2e825e23603f765757253ef115589be6			
Block No.	Sign&Pub cost	Block confirmation cost	Gas usage
7855424	1.375s	8.670s	22280

To further strengthen the auditability of malicious user behavior under a privacy-preserving approach, we introduce a lightweight blockchain logging mechanism. Specifically, whenever a user U sends a new prompt $\mathbf{x}$, we form a *log entry* that includes the user's pseudonymous identifier ID_U (or other suitable identifier). The current timestamp τ. A verification value $v_{\mathbf{x}}$, which may include the salted hash of the chunked embeddings (e.g., $H(\mathbf{v}_{x_i})$), or a combined hash of $\mathbf{x}$'s partial indicators. One practical design is: $\mathrm{LogHash} = H\Big(\mathrm{ID}_U \,\|\, \tau \,\|\, \mathrm{SomeVerificationTag}\Big)$. Upon creation, LogHash is then broadcast to a blockchain network as a simple transaction. Because blockchains provide an *immutable* and *append-only* ledger, this record cannot be retroactively altered without detection. Later, if a user is suspected of repeated prompt-injection attempts, the LLM service provider can reference the on-chain LogHash values at timestamps τ to confirm the *existence* and *sequence* of malicious prompts. Importantly, this process still does not expose the user's original plaintext prompts, as only the hash-derived LogHash is stored on-chain. By combining off-chain secure storage of $(H(\mathbf{x}), \mathbf{v}_x)$ with on-chain time-stamped proofs of submission, the system gains the benefits of *non-repudiation* and *tamper-evident* auditing, while fully respecting user privacy.

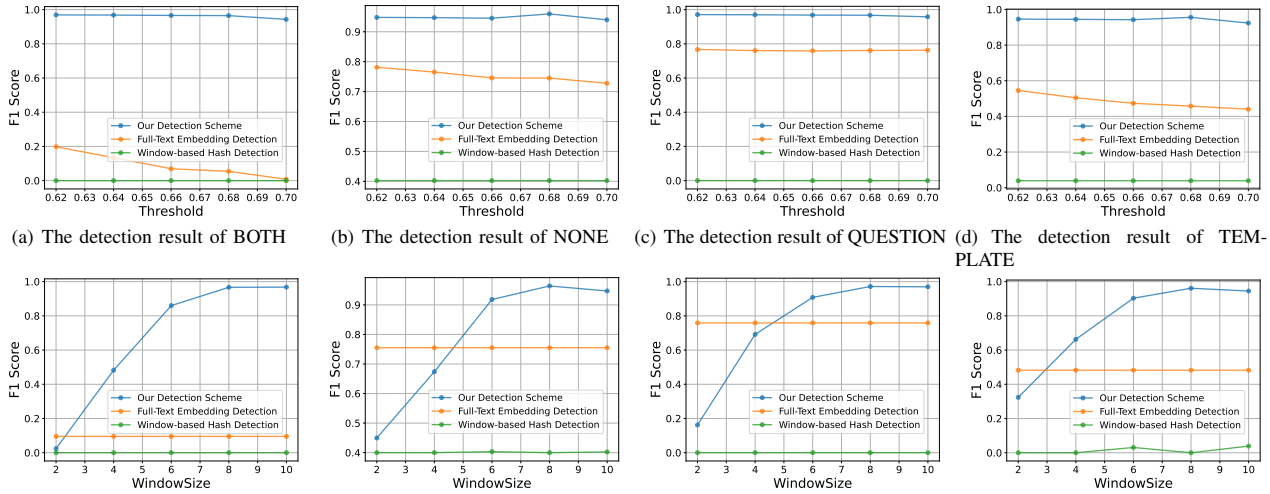
III. IMPLEMENTATION AND EVALUATION

We conduct a series of experiments to evaluate our privacy-preserving prompt injection detection scheme, assessing its effectiveness on multiple attack templates, questions, and normal prompts. Our testing framework integrates both local code and a lightweight blockchain setup for tamper-proof auditing. This section is divided into three parts: (i) implementation details and experimental environment, (ii) evaluation metrics, and (iii) experimental process and results.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENTAL SETTINGS OF LLM ATTACK DATASET

LLMs Attack Dataset	Attack Question Sets	Normal Question Sets	Test Models	Test Language
Ignore previous instructions [5]	50 Questions [14], [15] (Prohibited scenarios)	Random Generation	GPT-3.5-turbo/GPT-4o-mini	English
Information overload [5]	50 Questions [14], [15] (Prohibited scenarios)	Random Generation	GPT-3.5-turbo/GPT-4o-mini	English
Repeated-token attack [5]	50 Questions [14], [15] (Prohibited scenarios)	Random Generation	GPT-3.5-turbo/GPT-4o-mini	English
System mode [5]	50 Questions [14], [15] (Prohibited scenarios)	Random Generation	GPT-3.5-turbo/GPT-4o-mini	English



(a) The detection result of BOTH (b) The detection result of NONE (c) The detection result of QUESTION (d) The detection result of TEMPLATE

(e) The detection result of BOTH (f) The detection result of NONE (g) The detection result of QUESTION (h) The detection result of TEMPLATE

Fig. 2. Figures (a)-(d) show the detection results of the prompt under different vector similarity detection thresholds. Figures (e)-(h) show the detection results of the prompt under different scanning window sizes. We compared the detection scheme we proposed with full-text embedding detection scheme and window-based hash detection scheme.

A. Implementation Details and Environment

All experiments were performed on a MacBook Pro featuring an Apple M4 Max chip, 48 GB of RAM, and running macOS 15.3.1 (build 24D70). The primary experimental scripts use Python 3.9.21, with the following relevant packages: Faiss for fast similarity search on embeddings, nltk for tokenization, sentence-transformers as our embedding framework (default: all-MiniLM-L6-v2). The blockchain component is implemented in Node.js v20.11.0, primarily employing the ethers library. For testing, we connect to the Sepolia test network through Alchemy as our Ethereum node provider. This on-chain layer offers an immutable audit trail of user identity hashes and timestamps as shown in Table I. We adopt open-source prompt injection libraries (e.g., from [5], [14], [15]) to build a variety of injection templates. Table II summarizes the types of injection attacks and the associated question sets. Additionally, we create normal (benign) question sets for negative testing. For instance, we mix four injection types—"Ignore previous instructions," "Information overload," "Repeated-token attack," and "System mode"—each combined with 50 prohibited-scenario questions or randomly generated normal questions for baseline comparisons.

B. Evaluation Metrics

To quantify our scheme's performance, we track the following standard classification metrics: Precision (P): the fraction of prompts predicted to belong to a certain class (e.g., *BOTH*, *TEMPLATE*, etc.) that are actually correct. Recall (R): the fraction of true positives that our detection method successfully identifies. F1-Score: the harmonic mean of precision and recall, $F1 = 2 \times \frac{P \times R}{P+R}$. We compute F1 metrics for each of the four categories: *NONE*, *TEMPLATE*, *QUESTION*, and *BOTH*. In addition, we examine how these scores vary when adjusting parameters such as the similarity threshold (θ) for embedding-based detection or the window size (W) for sliding-window segmentation.

C. Experimental Process and Results

Figure 2 visualizes the effect of varying thresholds and window sizes on the F1-scores for each category. We generate 2000 test prompts by automatically combining injection templates, malicious questions, and normal questions into four classes (NONE, TEMPLATE, QUESTION, BOTH). We first fix the sliding-window size to 10 tokens and gradually increase the embedding similarity threshold θ from 0.62 to 0.70. As shown in the results, our sliding-window scheme consistently outperforms both a Full-Text Embedding De-

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.

tection approach and a Window-based Hash Detection approach. The F1-scores for our scheme remain above 0.94 across TEMPLATE and NONE classes, while BOTH and QUESTION also exhibit high accuracy, with minimal false positives. Meanwhile, the full-text embedding method lags noticeably, particularly in identifying BOTH or TEMPLATE, indicating that partial injection content is missed if not chunked. The hash-only detection is too rigid for paraphrased attacks, leading to very low recall in many categories. Next, we fix $\theta = 0.66$ and test window sizes ranging from 2 to 10 tokens. As the window size increases, the detection coverage improves dramatically, boosting F1-scores across all classes. Our sliding approach attains an F1 of up to 0.96 or higher for BOTH and NONE at $W = 8$ or $W = 10$, confirming that moderate chunk sizes balance local sensitivity and overall contextual capture. By contrast, too small windows ($W = 2$) hamper correct classification and yield lower recall, while hash-only detection again struggles with paraphrased content. Throughout these experiments, we optionally record user ID, timestamp, and hashed verification data on the Sepolia test network via Node.js scripts. Though not mandatory for the detection itself, this blockchain trace offers a tamper-proof log of suspicious prompt submissions, which can be audited later for repeated offenders or policy compliance checks.

Our sliding-window detection scheme achieves robust precision and recall for both exact and near-match injection prompts, while not storing any plaintext user data. These outcomes demonstrate that our design effectively safeguards privacy yet maintains accurate auditing and retroactive detection capabilities, even as new injection methods emerge. In conclusion, the experimental results confirm that *(i)* our chunk-based embedding approach yields high detection success against diverse prompt injection templates; *(ii)* tuning the threshold and window size can adapt the scheme to stricter or more lenient security policies; and *(iii)* incorporating blockchain logs for essential metadata strengthens the long-term integrity and accountability of the auditing process.

IV. Conclusion

In this work, we proposed a privacy-preserving prompt injection detection scheme tailored for large language models deployed in cloud environments. Our method segments each user prompt into chunks, applies salted hashing for exact match identification, and generates vector embeddings to capture semantically altered attacks. By storing only non-invertible representations of user text rather than raw content, the system balances security auditing needs with stringent privacy requirements. Additionally, an optional blockchain logging mechanism records essential metadata (such as user identity hashes and timestamps) on-chain, ensuring an immutable and tamper-evident audit trail without revealing sensitive user information. Experimental results demonstrate that our chunk-based detection approach significantly outperforms both full-text embedding and hash-only methods, achieving high F1 scores across multiple prompt types and thresholds. This dual-layer strategy effectively detects injection templates and malicious requests—whether verbatim or paraphrased—while mitigating the risk of data exposure. We also find that tuning parameters like chunk size and embedding similarity thresholds allows flexibility in striking the optimal balance between recall and precision. Future research directions include incorporating real-time updates for newly emerging injection patterns, enhancing embedding privacy (e.g., via differential privacy noise or homomorphic encryption).

Acknowledgments

This work was supported in part by the JSPS KAKENHI under Grants 23K11072 and 24KF0259, in part by the Telecommunications Advancement Foundation, and in part by the China Scholarship Council Program 202308050021.

References

[1] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, "A survey of knowledge enhanced pre-trained language models," *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[2] Y. Xu, L. Hu, J. Zhao, Z. Qiu, Y. Ye, and H. Gu, "A survey on multilingual large language models: Corpora, alignment, and bias," *arXiv preprint arXiv:2404.00929*, 2024.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.

[4] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng *et al.*, "Prompt injection attack against llm-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.

[5] M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil *et al.*, "Cyberseceval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv preprint arXiv:2404.13161*, 2024.

[6] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–39, 2025.

[7] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, and X. Cheng, "On protecting the data privacy of large language models (llms): A survey," *arXiv preprint arXiv:2403.05156*, 2024.

[8] Y. Hong, J. Wu, and X. Guan, "A survey of joint security-safety for function, information and human in industry 5.0," *Security and Safety*, vol. 4, p. 2024014, 2025.

[9] G. Hao, J. Wu, and S. Mumtaz, "Mobile e-health on-demand knowledge sharing with copyright protection: Joint blockchain and nft approach," in *ICC 2023 - IEEE International Conference on Communications*, 2023, pp. 6193–6198.

[10] G. Hao, Q. Pan, and J. Wu, "Incentive distributed knowledge graph market for generative artificial intelligence in iot," *IEEE Internet of Things Journal*, pp. 1–1, 2024.

[11] X. Xu, J. Wu, A. K. Bashir, and M. Omar, "Machine learning and zero knowledge empowered trustworthy bitcoin mixing for next-g consumer electronics payment," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 2210–2223, 2024.

[12] P. Rogaway and T. Shrimpton, "Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance," in *Fast Software Encryption: 11th International Workshop, FSE 2004, Delhi, India, February 5-7, 2004. Revised Papers 11*. Springer, 2004, pp. 371–388.

[13] F. Hassan, D. Sánchez, and J. Domingo-Ferrer, "Utility-preserving privacy protection of textual documents via word embeddings," *IEEE transactions on knowledge and data engineering*, vol. 35, no. 1, pp. 1058–1071, 2021.

[14] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan *et al.*, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *arXiv preprint arXiv:2204.05862*, 2022.

[15] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Jailbreaking chatgpt via prompt engineering: An empirical study," *arXiv preprint arXiv:2305.13860*, 2023.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:31:56 UTC from IEEE Xplore. Restrictions apply.